DEEP LEARNING · CNN ARCHITECTURES · STUDY NOTES

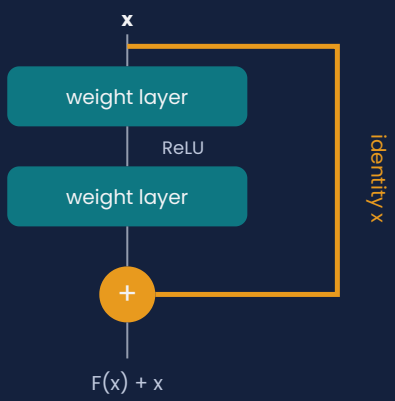
ResNet, Layer by Layer

A complete worked numerical example on ResNet-18: output sizes, parameters, MACs, and every new operation by hand. Every slide from *ResNet_Worked_Example.pptx*, explained from scratch.

He, Zhang, Ren & Sun · 2015

ILSVRC 2015 winner

ResNet-18: 11.7 M params · 1.81 G MACs



00 · Foundations
Residual block, BatchNorm, projection shortcut and the formulas used throughout.

01 · The problem and the big picture
Why deeper plain networks got worse, and ResNet-18 end to end.

02 · Part A: layer by layer
Stem, all four stages, the head, and the full tally of 11,689,512 parameters.

03 · Bottlenecks and the family
ResNet-50's 1×1 → 3×3 → 1×1 block, and ResNet-18 to ResNet-152.

04 · Part B: by hand
BatchNorm, residual addition, projection shortcut, gradient flow, why a block can do no harm.

05 · Wrap-up
Takeaways, the full trace, must-know numbers and self-test.

READ THIS FIRST · SLIDE 2

00 Foundations

ResNet adds three new pieces to an ordinary CNN.

Residual (skip) connection

The block's input **x** is carried around the layers and **added** to their output. The layers only have to learn the **change** F(x), not the whole output.

Batch normalization (BN)

After each conv, rescale each channel to mean 0 and variance 1 using the current mini-batch, then apply a learned scale γ and shift β. Makes deep networks train smoothly.

Projection shortcut

When the block changes shape (size halves, channels double), x is passed through a 1×1 conv with stride 2 so it matches F(x) and can be added.

The toolkit: formulas used throughout

OUTPUT SIZE

$$O = \lfloor (W - F + 2P) / S \rfloor + 1$$

Stride-2 3×3 convs (P = 1) halve 56 → 28 → 14 → 7.

CONV PARAMETERS (NO BIAS)

$$F \times F \times C_{in} \times K$$

ResNet convs have no bias: the BN that follows supplies it.

BATCH NORM PARAMETERS

$$2 \times K \quad (\gamma \text{ and } \beta)$$

Running mean and variance are stored but not learned.

MACS (CONV)

$$(O \times O \times K) \times (F \times F \times C_{in})$$

Number of outputs × work per output.

RESIDUAL BLOCK

$$y = \mathrm{ReLU}(F(x) + x)$$

The shortcut adds x back: no parameters, only additions.

PROJECTION SHORTCUT

$$1 \times 1 \times C_{in} \times C_{out} \text{ (stride 2)}$$

Used only where the shape changes, to make x match F(x).

W input width/height · **F** filter size · **P** padding · **S** stride · **C_in** input channels · **K** number of filters · ⌊ ⌋ = round down

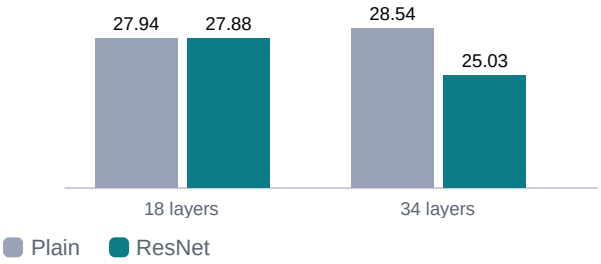
SLIDES 3–5

01 The problem, and ResNet-18 end to end

Slide 3 · Deeper plain networks got worse, not better

IMAGENET TOP-1 ERROR, % (LOWER IS BETTER)

	Plain network	ResNet
18 layers	27.94	27.88
34 layers	28.54 ↑	25.03 ↓



DEGRADATION

Plain-34 has higher error than plain-18, **and higher training error too**. So this is **not overfitting** (overfitting would mean good training error, bad test error): the deeper network is simply **harder to optimise**.

THE FIX

Let each block learn only a **residual F(x) = H(x) − x**, and add x back through a shortcut. H(x) is what the block should output overall.

THE RESULT

With shortcuts, depth helps again: **ResNet-34 beats ResNet-18 by 2.85 points** (27.88 − 25.03).

Numbers from Table 2 of He et al. (2015), 10-crop validation top-1 error. The same pattern appears on CIFAR-10 with 20 vs 56 plain layers.

Slide 4 · ResNet-18, end to end

Input	Conv1	MaxPool	conv2_x	conv3_x	conv4_x	conv5_x	Global AvgPool	FC + Softmax
224×224×3	7×7/2 112×112×64 + BN, ReLU	3×3/2 56×56×64	56×56×64 2 blocks	28×28×128 2 blocks	14×14×256 2 blocks	7×7×512 2 blocks	1×1×512	1,000

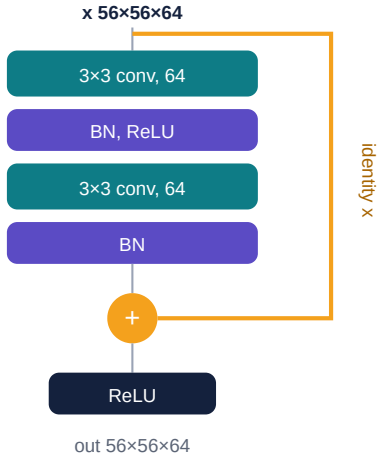
Convolution · Pooling · Residual stage · Fully connected

Why "18"? 1 (Conv1) + 4 stages × 2 blocks × 2 convs = 16, + 1 (FC) = **18 weight layers**. Each stage after the first **halves height and width and doubles the channels**.

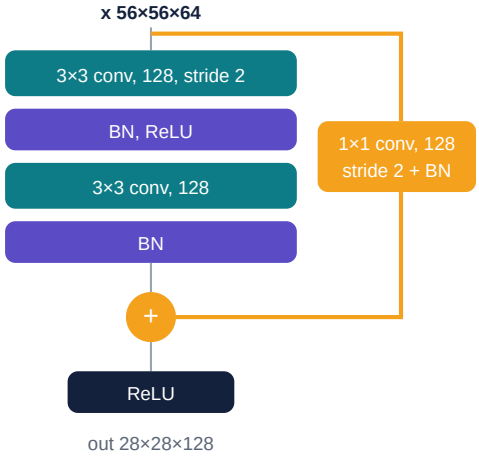
Slide 5 · The key idea: the basic residual block

DETAILS THAT GET ASKED

The addition happens **before the final ReLU**. The paper also tried zero-padding shortcuts (option A); torchvision uses the 1×1 projection (option B).



Identity shortcut (shape unchanged)



Projection shortcut (first block of a stage)

SLIDES 6–11 · PART A

02 Layer-by-layer calculation

Tracing ResNet-18 from the stem to the classifier.

Steps 1–2 · Stem: Conv1, BN and MaxPool

Input 224×224×3 · K = 64 · F = 7 · S = 2 · P = 3

Conv1

```
O = ⌊(224 − 7 + 6) / 2⌋ + 1 = 112
→ 112×112×64
Weights = 7 × 7 × 3 × 64 = 9,408  (no bias)
BN      = 2 × 64         =   128
Params  = 9,536
MACs    = 802,816 × 147 ≈ 118.0 M
```

MaxPool 3×3, S = 2, P = 1

```
O = ⌊(112 − 3 + 2) / 2⌋ + 1
  = ⌊55.5⌋ + 1 = 56
→ 56×56×64   after the stem
```

WHY NO BIAS?

BN subtracts the mean, which would cancel any bias, and then adds its own β. A conv bias would be wasted.

Step 3 · Stage conv2_x: two identity blocks at 56×56

Input 56×56×64 · every conv: 3×3, 64 filters, S = 1, P = 1 · size O = (56 − 3 + 2)/1 + 1 = 56

```
One conv
  Weights = 3 × 3 × 64 × 64 = 36,864
  MACs    = 3,136 × 64 × 576 ≈ 115.6 M
One block (2 convs + 2 BN)
  Params = 2 × 36,864 + 2 × 128 = 73,984
  MACs   = 2 × 115.6 M          = 231.2 M
Stage (2 blocks)
  Params = 147,968      MACs ≈ 462.4 M

3,136 = 56 × 56 output positions;
576 = 3 × 3 × 64 MACs each.
```

The shortcut

```
Shapes match (56×56×64 in and out),
so the shortcut is the identity.
Params     = 0
MACs       = 0
Additions  = 56×56×64 = 200,704
```

ESSENTIALLY FREE

Additions are about **0.1%** of the block's 231 M MACs.

Step 4 · Stage conv3_x: downsampling with a projection

Path	Layer	Output size	Weights	BN	MACs
Main	3×3, 128, S=2	⌊(56−3+2)/2⌋+1 = 28	73,728	256	57.8 M
	3×3, 128, S=1	28	147,456	256	115.6 M
Shortcut	1×1, 128, S=2	⌊(56−1)/2⌋+1 = 28	8,192	256	6.4 M
Block 1		**28×28×128**	**229,376**	**768**	**179.8 M**

WHY A PROJECTION?

F(x) is 28×28×128 but x is 56×56×64, so they cannot be added. A **1×1 conv with stride 2 fixes both mismatches at once**, at a cost of only 8,192 weights.

```
Block 1   params = 229,376 + 768 = 230,144
Block 2   2×147,456 + 512 = 295,424
          MACs = 231.2 M
Stage     525,568 params, 411.0 M MACs
```

Steps 5–6 · Stages conv4_x and conv5_x

Sizes: ⌊(28−3+2)/2⌋+1 = 14 and ⌊(14−3+2)/2⌋+1 = 7.

Block	Output	Conv a	Conv b	Shortcut	BN	Params	MACs
conv4_x b1	14×14×256	3×3×128×256 = 294,912	3×3×256×256 = 589,824	1×1×128×256 = 32,768	1,536	919,040	179.8 M
conv4_x b2	14×14×256	589,824	589,824	identity	1,024	1,180,672	231.2 M
conv5_x b1	7×7×512	3×3×256×512 = 1,179,648	3×3×512×512 = 2,359,296	1×1×256×512 = 131,072	3,072	3,673,088	179.8 M
conv5_x b2	7×7×512	2,359,296	2,359,296	identity	2,048	4,720,640	231.2 M

2,099,712
conv4_x params · 411.0 M MACs

8,393,728
conv5_x params · 411.0 M MACs

Steps 7–8 · Global average pooling and the classifier

Head

```
AvgPool 7×7: 7×7×512 → 1×1×512
Params = 0

FC 512 → 1000
  512×1000 + 1000 = 513,000
  MACs = 512,000 ≈ 0.5 M

Softmax → 1000 probabilities
```

Compare: flattening instead

```
Flatten: 7×7×512 = 25,088
FC → 1000: 25,088,000 + 1,000
         = 25,089,000 params

GAP + FC:  513,000 params
```

WHY GAP

A flattened head alone would have **more than twice the parameters of the entire ResNet-18**. Also: **no dropout**, because batch norm already regularises the network.

Step 9 · The full tally

Layer	Output shape	Parameters	MACs
Conv1 + BN	112×112×64	9,536	118.0 M
MaxPool	56×56×64	0	–
conv2_x (2 blocks)	56×56×64	147,968	462.4 M
conv3_x (2 blocks)	28×28×128	525,568	411.0 M
conv4_x (2 blocks)	14×14×256	2,099,712	411.0 M
conv5_x (2 blocks)	7×7×512	8,393,728	411.0 M
Global AvgPool	1×1×512	0	–
FC	1000	513,000	0.5 M
Total		**11,689,512**	**≈ 1.81 G**

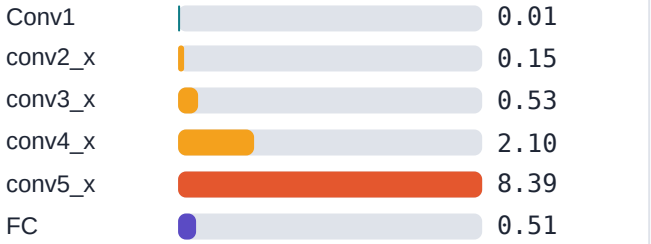
11.7 M
parameters

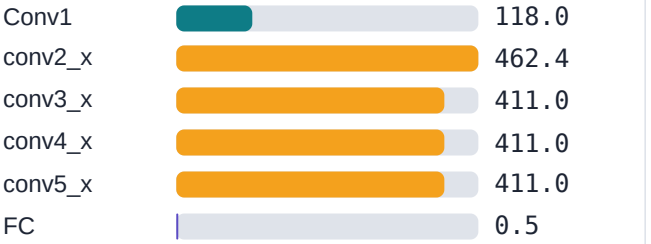
1.81 G
multiply-accumulates

≈ 47 MB
weights at FP32

This matches the parameter count of torchvision's resnet18 exactly. Params include BN γ and β; conv layers have no bias. The paper quotes 1.8 billion multiply-adds for ResNet-18.

Slide 12 · The key insight: compute is spread evenly; parameters pile up at the end





WHY EACH LATER STAGE COSTS EXACTLY 411.0 M MACS

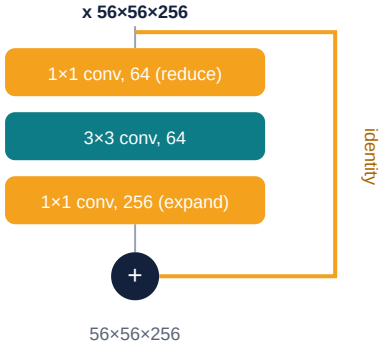
Moving to the next stage divides O × O by 4 (height and width halve) but doubles both C_in and K, which multiplies K × C_in by 4. The two cancel. **conv5_x holds 72% of the parameters** at only 7×7 resolution.

SLIDES 13–14

03 ResNet-50 and deeper

The bottleneck block, and the whole family from 18 to 152 layers.

Slide 13 · The bottleneck block



Bottleneck on 256 channels

```
1×1: 256 × 64       =  16,384
3×3: 3 × 3 × 64×64  =  36,864
1×1: 64 × 256       =  16,384
Weights             =  69,632
MACs ≈ 218.4 M

Each conv is followed by BN (and
ReLU, except after the last one).
```

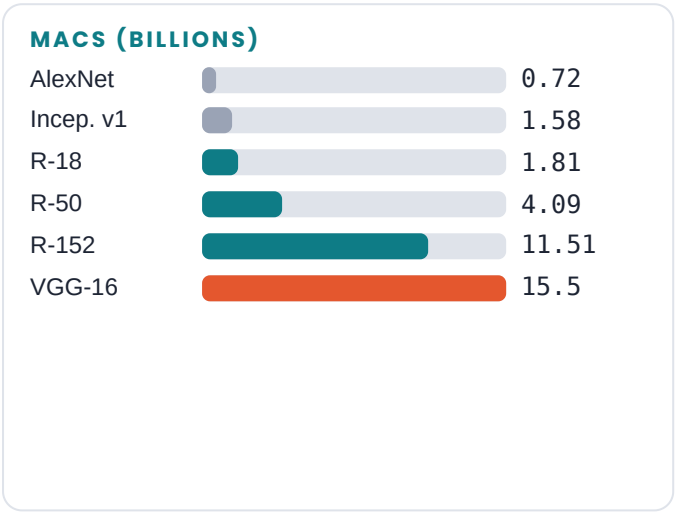
Block type	Width	Weights	MACs at 56×56
Bottleneck	**256**	**69,632**	**218.4 M**
Basic (ResNet-18/34)	64	73,728	231.2 M
Basic, if made 256 wide	256	1,179,648	3.70 G

SAME TRICK AS INCEPTION

1×1 convs reduce depth before the expensive 3×3. **A bottleneck block is 4× wider than a basic block for about the same cost.** (Weights exclude BN.)

Slide 14 · The family: from ResNet-18 to ResNet-152

Model	Block	Blocks per stage	Params	MACs
ResNet-18	basic	2, 2, 2, 2	11.7 M	1.81 G
ResNet-34	basic	3, 4, 6, 3	21.8 M	3.66 G
ResNet-50	bottleneck	3, 4, 6, 3	25.6 M	4.09 G
ResNet-101	bottleneck	3, 4, 23, 3	44.5 M	7.80 G
ResNet-152	bottleneck	3, 8, 36, 3	60.2 M	11.51 G



LAYER COUNT

For bottleneck models: **3 × (sum of blocks) + 2**.
ResNet-50: 3 × (3+4+6+3) + 2 = 3 × 16 + 2 = 50.

152 LAYERS, LESS COMPUTE THAN VGG-16

Bottlenecks and global average pooling keep ResNet-152 at **11.5 G MACs versus VGG-16's 15.5 G**, with **60 M parameters against 138 M**.

These counts, computed with the formulas above, match torchvision (whose ResNet-50+ puts the stride on the 3×3 conv, "v1.5"). VGG-16 figures are the commonly cited 138 M params and 15.5 G MACs.

SLIDES 15–20 · PART B

04 Every new operation, by hand

Convolution, pooling and softmax work as before. Here we compute what ResNet adds, on small numbers.

B1 · Batch normalization: one channel, one position, a mini-batch of 4 images

```
Input x = [2, 4, 6, 8]

μ  = (2 + 4 + 6 + 8) / 4 = 5
σ² = (9 + 1 + 1 + 9) / 4 = 5
σ  = √5 ≈ 2.236   (ε ≈ 0 here)

x̂ = (x − μ) / σ:  (2 − 5)/2.236 = −1.342
Normalized x̂ = [−1.342, −0.447, 0.447,
1.342]

y = γx̂ + β   (γ = 2, β = 1)
  2·(−1.342) + 1 = −1.683
Output y = [−1.683, 0.106, 1.894, 3.683]
```

WHAT BN LEARNS

γ and β are learned, one pair per channel. After normalising to mean 0 and variance 1, BN lets the network rescale to whatever suits it.

Training vs test

Running mean and variance are tracked during training and used at test time, when there is no batch to average over.

Count in ResNet-18

BN follows every conv: **9,600** learnable BN parameters in total (2 × channels summed over all 20 BN layers: 17 main-path convs + 3 projections).

μ = mean, σ² = variance (average squared distance from the mean), ε = a tiny number to avoid dividing by zero.

B2 · Residual addition: y = ReLU(F(x) + x)

SHORTCUT X

1	−2
3	0

+

F(X) FROM THE CONVS

0.5	1
−4	2

=

F(X) + X

1.5	−1
−1	2

→

RELU

1.5	0
0	2

```
(0,0):  0.5 + 1    = 1.5 → 1.5
(0,1):  1   + (−2) = −1  → 0
(1,0): −4   + 3    = −1  → 0
(1,1):  2   + 0    = 2   → 2
```

ELEMENT-WISE

The shapes must match exactly. The ReLU comes **after** the addition, so the shortcut can carry negative values into the sum.

B3 · Projection shortcut: 1×1 conv, stride 2, 4×4×1 → 2×2×2

INPUT X (4×4, ONE CHANNEL)

1	5	**2**	7
3	0	4	1
6	2	**8**	3
0	9	1	4

Stride 2 with a 1×1 filter visits only the amber pixels.

```
O = ⌊(4 − 1)/2⌋ + 1 = 2
Sampled: [[1, 2], [6, 8]]

Filter 1: w = 2   → [[2, 4], [12, 16]]
Filter 2: w = −1  → [[−1, −2], [−6,
−8]]

Weights = 1×1×1×2 = 2
```

OUTPUT CH 1

2	4
12	16

OUTPUT CH 2

−1	−2
−6	−8

SHAPES NOW MATCH

x has the same 2×2×2 shape as F(x), so they can be added. In ResNet-18 conv3_x: 1×1×64×128 = **8,192** weights turn 56×56×64 into 28×28×128.

B4 · Gradient flow: the "+ x" gives the gradient a direct path

$$y = x + F(x) \Rightarrow \partial L/\partial x = \partial L/\partial y \cdot (1 + \partial F/\partial x)$$

$\partial L/\partial x$ is how much the loss changes when x changes. The **1** comes from the shortcut: its derivative is always 1.

Plain block y = F(x)

$\partial L/\partial y = 2.0, \quad \partial F/\partial x = 0.1$

One block: $2.0 \times 0.1 = 0.2$

Ten blocks: $2.0 \times 0.1^{10} = 2 \times 10^{-10}$

The gradient vanishes.

Residual block y = x + F(x)

$\partial L/\partial y = 2.0, \quad \partial F/\partial x = 0.1$

One block: $2.0 \times (1 + 0.1) = 2.2$

Ten blocks: $2.0 \times 1.1^{10} \approx 5.19$

The "1" path alone delivers 2.0 intact.

Simplified scalar example: real Jacobians are matrices, but the identity term still guarantees an undiminished path back to early layers.

B5 · Why a block can do no harm: if F learns nothing, the block passes x through

Residual block with F(x) = 0

```
x    = [1.5, 0, 2.0, 0.7]   (≥ 0 after ReLU)
F(x) = [0,   0, 0,   0  ]
y    = ReLU(x + 0)
     = [1.5, 0, 2.0, 0.7]  = x ✓
```

PLAIN LAYER MUST LEARN THE IDENTITY

0	0	0
0	**1**	0
0	0	0

For a 3×3 conv to copy its input, every filter needs exactly this kernel on its own channel and zeros elsewhere. Hard to hit exactly from random initialization, through several stacked layers.

THE CONSEQUENCE

The optimiser only has to drive F's weights toward zero, which is easy. So **a deeper ResNet can always match a shallower one by switching extra blocks to identity**, and adding depth should not raise the training error. That is exactly the degradation problem, solved. (torchvision's `zero_init_residual` option starts each block this way by setting the last BN's γ to 0.)

SLIDE 21 · WRAP-UP

05 What to take away

Learn the residual

Each block learns F(x) = H(x) − x and adds x back. Identity shortcuts cost 0 parameters.

Gradients get a highway

$\partial L/\partial x = \partial L/\partial y\,(1 + \partial F/\partial x)$: the "1" stops gradients vanishing through 100+ layers.

Cheap, deep, and even

11.7 M params, 1.81 G MACs for ResNet-18. Later stages cost the same 411 M MACs each.

THE FULL TRACE FOR ONE IMAGE (RESNET-18)

224×224×3 → 112×112×64 → 56×56×64 → 56×56×64 → 28×28×128 → 14×14×256 → 7×7×512 → 1×1×512 → 1000 probabilities

Must-know numbers

Fact	Number
Degradation evidence (top-1 error)	Plain 18 → 34: 27.94 → 28.54 (worse) · ResNet 18 → 34: 27.88 → 25.03 (2.85 better)
Why 18	1 + 4 × 2 × 2 + 1 = 18 weight layers
Stem	Conv1 9,536 params (9,408 + 128 BN), 118.0 M MACs → MaxPool → 56×56×64
Stage params	147,968 · 525,568 · 2,099,712 · 8,393,728
Stage MACs	462.4 M, then 411.0 M each
Projection in conv3_x	1×1×64×128 = 8,192 weights
Head	GAP + FC = 513,000 vs flatten 25,089,000
Total ResNet-18	11,689,512 params · ≈ 1.81 G MACs · ≈ 47 MB · 9,600 BN params
Bottleneck (256 wide)	69,632 weights · 218.4 M MACs vs 1,179,648 · 3.70 G basic
ResNet-50 / 152	25.6 M · 4.09 G / 60.2 M · 11.51 G (VGG-16: 138 M · 15.5 G)

Test yourself

Q1. Plain-34 has higher *training* error than plain-18. Why does that rule out overfitting?

Q2. Why do ResNet convs have no bias?

Q3. When is a projection shortcut used, and what does it cost in conv3_x?

Q4. Why does every stage after conv2_x cost the same 411 M MACs?

Q5. With $\partial F/\partial x = 0.1$ per block, what is the gradient through 10 residual blocks if $\partial L/\partial y = 2$?

A1. Overfitting means fitting training data too well. Here the deeper net fits the training data worse, so it is an optimisation problem.

A2. The BN after each conv subtracts the mean (cancelling any bias) and adds its own β.

A3. Only when the shape changes. 1×1×64×128 = 8,192 weights (+ 256 BN).

A4. O × O falls by 4 while C_in × K rises by 4; they cancel.

A5. $2.0 \times 1.1^{10} \approx 5.19$ (vs 2×10^{-10} for plain blocks).